Page No.:
Date:

2. Training, Optimization & Regularization of DNN

Q.1 What is the significance of activation function? and explain different types of activation function.

Ans)

- The main aim of activation function is to introduce non-linearity in the neural network.
- Activation function convert this linear input signals of a node into non-linear output signals to facilitate the learning of high order polynomials.
- A distinct property of activation function is that they are differentiable.
- This property helps them function during the backpropagation of the neural networks.
- Significance :

1) The activation function in neural network decides whether a neuron should be activated or not by calculating weighted sum and adding bias.
2) Without activation functions, a neural network would behave like a simple linear regression model, no matter how many layers it has.
3) It introduces non-linearity into the model, allowing the network to learn complex pattern.

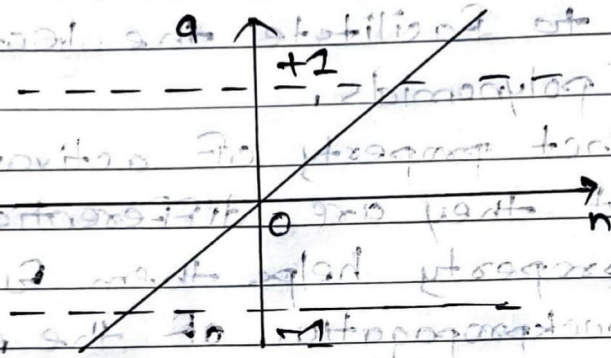
Page No.:
Date:

Types

1) Linear

⇒ Linear activation function gives the output same as that of the input or net value.

- The Matlab toolbox has a function, purelin, to realize the mathematical linear transfer function shown below:



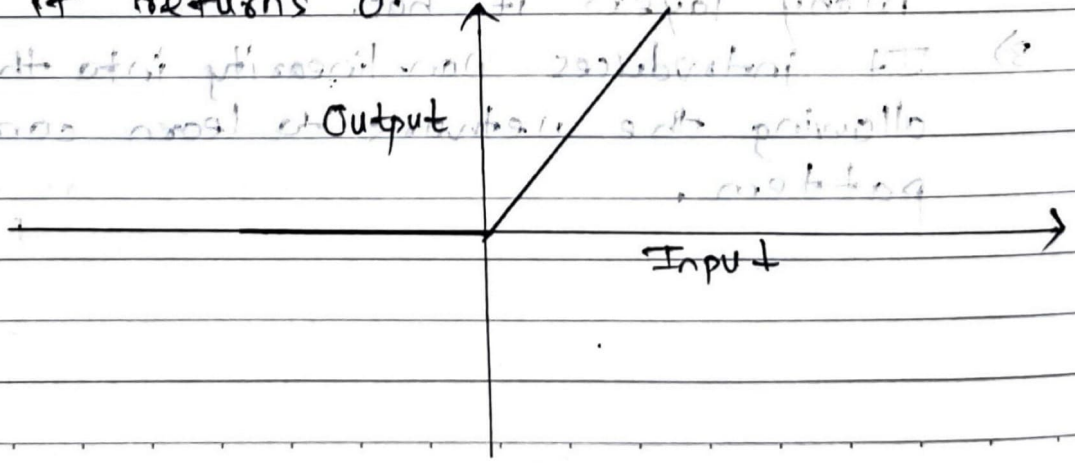
a = purelin(n)

- The equation is

Output = net.

2) Rectified Linear Unit (ReLU)

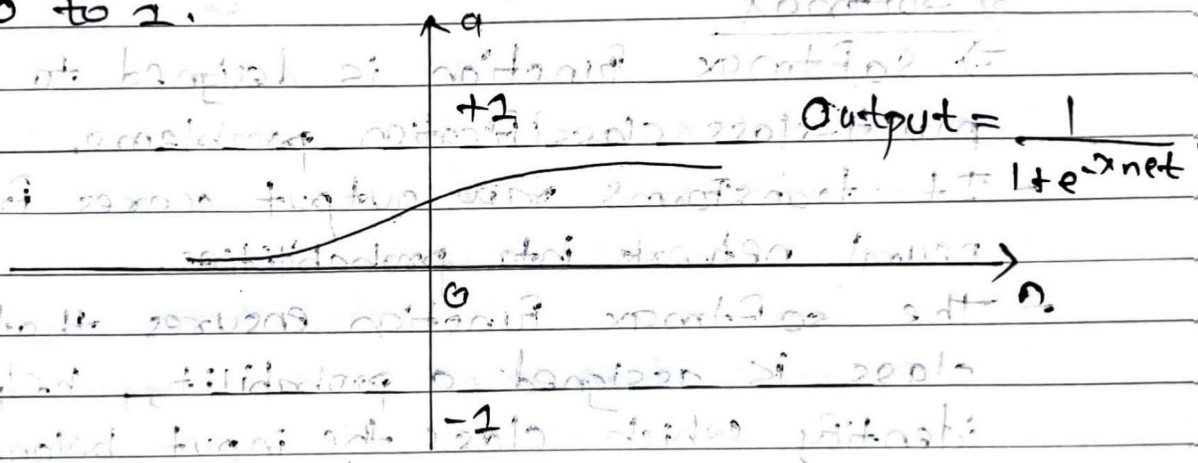
⇒ ReLU activation is defined. A(x) = max(0,x) this means that if input x is positive, ReLU returns x, if input is negative, it returns 0.



- ReLU shows better generalization and performance when compared to other activation functions like the sigmoid and tanh functions.
- Value range : $[0, \infty)$

3) Logistic / Unipolar Continuous

=> The sigmoid transfer function shown below takes the input, which can have any value between plus and minus infinity, and squashes the output into the range 0 to 1.



$a = logsig(n)$

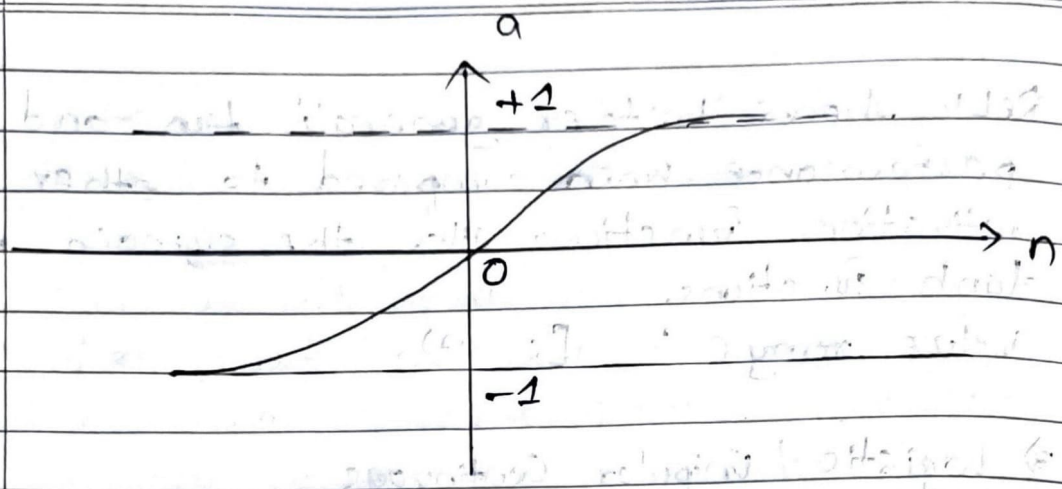
- This transfer function is commonly used in back propagation networks, in parts because it is differentiable.

4) tanh / Bipolar continuous

=> Tanh function is a shifted version of sigmoid, allowing it to stretch across the y-axis.

- It is defined as :-

$$Output = \left(\frac{2}{1+e^{-x net}}\right) - 1$$



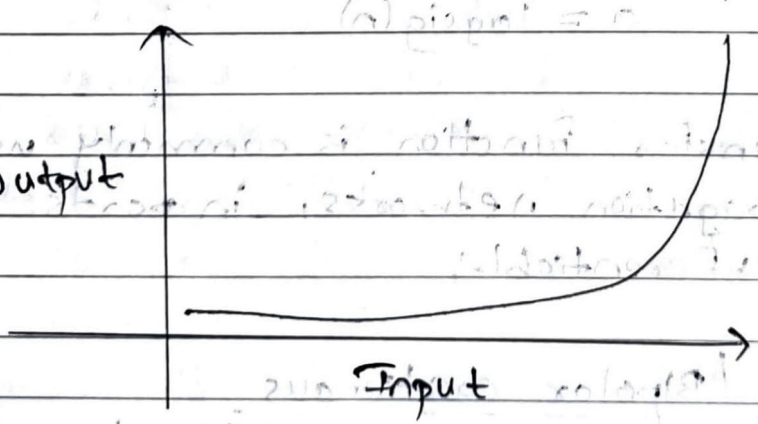
$a = tansig(n)$

- Value range : -1 to +1

5) Softmax

⇒ Softmax function is designed to handle multi-class classification problems.

- It transforms raw output scores from a neural network into probabilities.
- The softmax function ensures that each class is assigned a probability, helping to identify which class the input belongs to.



- Range : 0 to 1.
- Equation :

$$Softmax(z_i) = \frac{\exp(z_i)}{\Sigma \exp(z_i)}$$

Page No.:
Date:

Q.2 What is significance of loss function? Explain different types.

Ans

- loss function helps you figure out the performance of your model in prediction, how good the model is able to generalize.
- It computes the error for every training.
- The loss function is the function calculating the distance between the current output and the expected output of the algorithm.
- Significance :-

1) The loss function measures how well a neural network's predicted outputs match the actual target values.
2) It calculates the error between the predicted output and truth output.
3) The goal of training a neural network is to minimize this loss, helping the model learn better patterns from the data.
4) Without a loss function, the model would have no way to know how far off it's predictions are or how to improve

- Types :-
  - 1) MSE
  - 2) Cross-entropy loss
  - 3) MAE
  - 4) Huber loss

Page No.:
Date:

1) Mean Squared Error (MSE)

⇒ MSE is calculated as the average of the squared differences between the estimated and actual values.

- The result is always positive regardless of the sign of the predicted and actual values and a perfect value is 0.0
- The squaring means that larger mistakes result in more error than smaller mistakes, meaning that the model is punished for making larger mistakes.
- The MSE function is expressed as -

$$MSE = \frac{1}{N} \sum_{i=1}^{N} ||actual_i - predicted_i||^2$$

- For e.g.,

X	Y	Error	Squared Error
10	12	-2	4
8	8	0	0
20	24	-4	16
13	14	-1	1
Sum			21

$$MSE = \frac{21}{4} = 5.25$$

Page No.:
Date:

2) Cross-entropy loss function

- Also called as logarithmic loss, log loss or logistic loss.
- Each predicted class probability is compared to the actual class desired output 0 or 1 and a score/loss is calculated that penalizes the probability based on how far it is from the actual expected value.
- Cross-entropy loss is used when adjusting model weights during training.
- The aim is to minimize the loss, i.e., the smaller the loss better the model.
- A perfect model has a cross-entropy loss of 0.
- Cross-entropy is defined as -

$$L_{CE} = -\sum_{i=1}^{n} t_i \cdot \log(p_i)$$

where,

t_i = truth label,

p_i = softmax probability

- For e.g.,

S	t
0.775	1
0.116	0
0.039	0
0.070	0

$$L_{CE} = -\left[1 * \log_2(0.775) + 0 * \log_2(0.116) + 0 * \log_2(0.039) + 0 * \log_2(0.070)\right]$$

$$= 0.3677$$

Page No.:
Date:

3) Mean Absolute Error (MAE)

⇒ MAE another loss function used in regression tasks.

- It computes the average of the absolute differences between the predicted values and actual values.
- Formula :-

$$L_{MAE} = \frac{1}{N} |y_i - \hat{y}_i|$$

4) Huber loss

⇒ The Huber loss function is a combination of MSE and MAE, designed to take advantage of best properties of both loss functions.

- It behaves like MSE when the error is small and like MAE when the error is large.
- This makes it less sensitive to outliers than MSE.
- The formula:

$$\text{Loss} = \begin{cases} \frac{1}{2}(x-y)^2, & \text{if } |x-y| \le \delta \\ \delta \cdot |x-y| - \frac{1}{2}\delta^2, & \text{otherwise} \end{cases}$$

Page No.:
Date:

Q.3 Explain optimization. Discuss SGD and Adam optimization algorithms.

⇒

- Optimisation refers to the process of adjusting the weights and biases of neural network to minimize the loss function.
- The objective is to find the best set of parameters that allow the model to make accurate predictions on unseen data.
- This is done using optimization algorithms that iteratively update model parameters based on gradients computed through backpropagation.

1) Stochastic Gradient Descent (SGD)

⇒ In batch gradient descent we were considering all the examples for every step of Gradient descent.

- But what if our dataset is very huge. Deep learning models crave for data.
- The more the data the more chances of model to be good.
- Suppose our dataset has 5 millions example, then just to take one step the model will have to calculate the gradients of dl the 5 millions examples.
- This does not seem an an efficient way.
- To tackle this problem, we have SGD.
- In SGD, we consider just one example at a time to take a single step.

- We do the following steps in one epoch for SGD:
  a) Take an example
  b) Feed it to Neural Network
  c) Calculate its gradient
  d) use the gradient we calculated in step 3 to update the weights.
  e) Repeat steps 1-4 for all examples in training dataset.
- Since we are considering just one example at a time the cost will fluctuate over the training examples and it will not necessarily decreases.
- SGD can be used for larger datasets.
- It converges faster when the dataset is large as it causes updates to the parameters more frequently.
- The update rule is

$$\theta = \theta - \alpha \nabla J_i(\theta),$$

Where, θ = parameter vector,
α = Hyperparameter
$\nabla J_i(\theta)$ = Gradient

Page No.:
Date:

2) Adam GD

=) Adaptive Moment (Adam) is one of the most effective optimization algorithms for training neural networks.

- It combines ideas from RMSProp and Momentum.
- It calculates an exponentially weighted average of past gradients, and it stores it in variables v and v-corrected.
- The update rules :-

$$V_t = \beta_1 V_{t-1} + (1-\beta_1) \nabla J(\theta)$$
$$G_t = \beta_2 G_{t-1} + (1-\beta_2) \nabla J(\theta)^2$$

$$\hat{V}_t = \frac{V_t}{1-\beta_1^t}$$
$$\hat{G}_t = \frac{G_t}{1-\beta_2^t}$$

$$\theta = \theta - \frac{\alpha}{\sqrt{\hat{G}_t} + \varepsilon} \cdot \hat{V}_t$$

where,

α = learning rate (0.001)

β_1 & β_2 = Decay rates

$\beta_1 = 0.9$, $\beta_2 = 0.999$

ε = small positive constant (10^{-8}) used to avoid division by zero when computing the final update.

Page No.:
Date:

Q.4 What are different types of gradient descent methods, explain any three.

⇒

- Gradient descent is an optimization technique used to minimize a loss function and update the model parameters.
- There are three main types of GD method based on how much data is used to compute the gradient in each update:
  - 1) Batch (Vanilla) GD
  - 2) Stochastic GD
  - 3) Mini-batch GD.

1) Batch GD

⇒ In Batch Gradient Descent, all the training data is taken into consideration to take a single step.

- We take the average of the gradients of all the training examples and then use that mean gradient to update our parameters.
- So that's just one step of gradient descent in one epoch.
- The update rule:

$$\theta = \theta - \alpha \cdot \nabla J(\theta),$$

Where,

θ = parameter vector,

α = Hyperparameter

$\nabla J(\theta)$ = gradient ~~descent~~ of cost function J w.r.t θ

- Batch gradient is great of convex or relatively smooth error manifolds.
- In this case, we move somewhat directly towards an optimum solution.

2) Stochastic GD

⇒ Refer Q. 3

3) Mini-Batch GD

⇒ Batch gradient descent can be used for smoother curves. SGD can be used when the dataset is large.

- Batch gradient descent converges directly to minima. SGD converges faster for larger datasets.
- But, since in SGD we use only one example at a time, we cannot implement the vectorized implementation on it.
- This can slow down the computations.
- To tackle this problem, a mixture of Batch GD and SGD is used.
- Neither we use all the dataset all at once nor we use the single example at a time.
- We use a batch of fixed number of training examples which is less than the actual dataset and call it a mini-batch.

- So, after creating the mini-batches of fixed size, we do the following steps in one epoch:

a) Pick a mini-batch
b) Feed it to Neural Network
c) Calculate the mean gradient of mini-batch
d) Use the mean gradient we calculated in step 3 to update the weights.
e) Repeat steps 1-4 for the mini-batches we created.

- The update Rule:

$$\theta = \theta - \alpha \cdot \frac{1}{m} \sum_{i=1}^{m} \nabla J_i(\theta)$$

Page No.:
Date:

Q.3 Explain GD optimization techniques.

⇒

- Gradient descent is a fundamental optimization algorithm used to minimize the loss function.
- It works by iteratively updating the model parameters in the opposite direction of the gradient of the loss function w.r.t parameters.
- The goal is to find set of the parameters where the loss function is at its minimum.
- Update formula:

$$\theta = \theta - \alpha \cdot \Delta J(\theta).$$

- Types:-
  - 1) Batch GD
  - 2) SGD
  - 3) Mini-batch GD
  - 4) Momentum GD
  - 5) Nesterov GD
  - 6) AdaGrad GD
  - 7) RMSProp GD
  - 8) Adam GD

1) Momentum-based GD

⇒ Standard gradient descents can be slow and may get stuck in local minima.

- Momentum adds a fraction of previous updates to the current update, helping model move faster and escape local minima.

- Update Rules :-

$$V_t = \beta_{t-1} + (1-\beta)\nabla J(\theta)$$

where,

V_t = velocity,

β = momentum factor,

α = learning rate,

$\nabla J(\theta)$ = Gradient.

- Advantages :

i) Faster convergence

ii) Reduces oscillation.

2) Nesterov Accelerator Gradient (NAG)

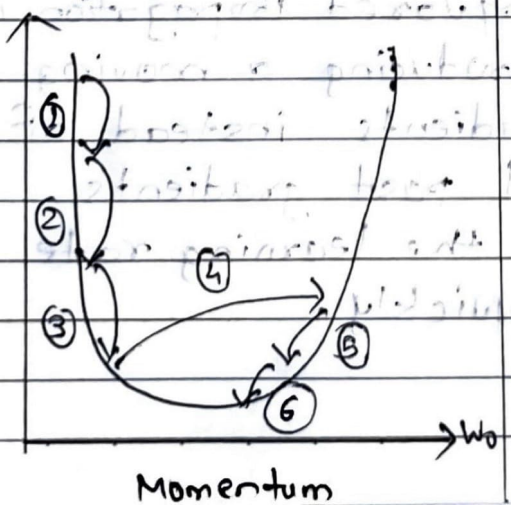
⇒ NAG is an improvement over the momentum-based GD.

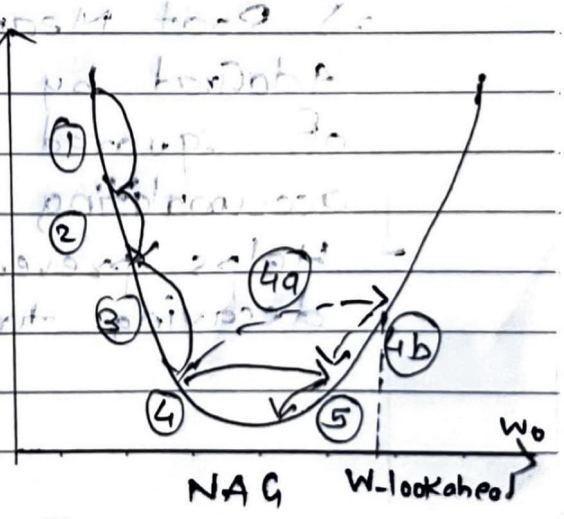
- Instead of using the previous gradient directly, Nesterov computes a look-ahead step and adjusts based on future information.

- Update rules :-

$$V_t = \beta V_{t-1} + (1-\beta)\nabla J(\theta - \beta V_{t-1})$$

$$\theta = \theta - \alpha . V_t$$





Page No.:
Date:

- NAG is considered more efficient than classical momentum because it has a better understanding of the future trajectory, leading to even faster convergence and better performance in some cases.

3) Adagrad

=> Adagrad adapts the learning rate for each parameter individually.

- Parameters with high gradient get lower learning rate, while parameters with smaller gradient gets higher learning rate.
- Update Rules

$$G_t = G_{t-1} + \nabla J(\theta)^2$$

$$\theta = \theta - \frac{\alpha}{\sqrt{G_t} + \varepsilon} \nabla J(\theta)$$

where,

G = Sum of squared gradients

ε = Constant

4) RMSProp

=> Root Mean Squared Propagation improves AdaGrad by introducing a moving average of squared gradients instead of accumulating all past gradients.

- Helps prevent the learning rate from decaying too quickly.

- Update Rule :-

$$G_t = \beta \cdot G_{t-1} + (1-\beta)\, \nabla J(\theta)^2$$

$$\theta = \theta - \frac{\alpha}{\sqrt{G_t + \epsilon}} \cdot \Delta J(\theta)$$

Page No.:
Date:

Q.6 What are L_1 and L_2 regularization?

⇒

- Regularization is a set of strategies used in ML to reduce the generalization error.
- Most models, after training, perform very well on a specific subset of the overall population but fail to generalize well.
- This is also known as overfitting.
- Regularization strategies aims to reduce overfitting and keep, at the same time, the training error as low as possible.

1) L_1 Regularization

⇒ A regression model which uses the L_1 regularization technique is called LASSO regression.

- Lasso regression adds the absolute value of magnitude of the coefficients as a penalty term to the loss function.
- Lasso regression also helps us achieve feature selection by penalizing the weights to approximately equal to zero if that feature does not serve any purpose in the model.
- The equation :-

$$\text{cost} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} |w_i|$$

where,

m = no. of feature,

n = no. of examples,

Page No.
Date:

y_i = Actual target value;
$\hat{y}_i$ = Predicted target value,

- L1 regularization is particularly useful when working with high-dimensional data since it enables one to choose a subset of most important attributes.
- This lessens the risk of overfitting and also makes the model easier to understand.
- The size of penalty term is controlled by the hyperparameter lambda, which regulates the L1 regularization's regularization strength.
- As lambda rises, more parameters will be lowered to zero, improving regularization.

2) L2 Regularization

⇒ A regression model that uses the L2 regularization technique is called Ridge regression.

- Ridge regression adds the squared magnitude of the coefficient as a penalty term to the loss function.
- The equation:-

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} w_i^2$$

Where,
n = no. of examples,
m = no. of features,
y_i = Actual target value for i^{th} example,
$\hat{y}_i$ = Predicted target value —11—

Page No.:
Date:

w_i = coefficients of the features
λ = Regularization parameter.

- A hyperparameter called lambda that controls the regularization's intensity also control the size of penalty term.
- The parameters will be smaller and the regularization will be stronger the greater the lambda.

Page No.:
Date:

Q.7 Explain early stopping, Batch normalization and Data augumentation.

⇒

1) Early Stopping :

- Early stopping is one of the most commonly used strategies because it is very simple and quite effective.
- It refers to the process of stopping the training when the training error is no longer decreasing but the validation error is starting to rise.
- This implies that we store the trainable parameters periodically and track the validation error.
- After the training stopped, we return the trainable parameter to the exact point where the validation error started to rise, instead of last ones.
- A different way to think of early stopping is as very efficient hyperparameter selection algorithm, which sets the no. of epochs to the absolute best.
- Benefits:

i) Improves generalization
ii) Saves training time and avoids unnecessary computation.

Page No.:
Date:

2) Batch Normalization

⇒ Batch normalization is a technique used in DL to make neural networks train faster and more stable by normalizing the inputs to each layer.

- Batch Normalization helps by:
  a) Stabilizing learning
  b) Speeding up training
  c) Allowing higher learning rates.
  d) Regularizing the model.

- For a given mini-batch during training, Batch Norm:

a) Computes the mean and variance:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

b) Normalizes the inputs:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

c) Scale and shifts the normalized value using learnable parameters:

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

Page No.
Date

3) Data Augumentation

⇒ Data augumentation is a technique to artificially increase the size and diversity of training dataset by applying transformation to existing data.

- Data augumentation can be achieved in many different ways.

i) Basic data manipulation : The first simple thing to do is to perform geometric transformation on data.

- Most notably, if we're talking about images we have solutions such as Image Flipping, cropping, rotations, translation, image color modification, etc.

ii) Feature space augumentation : Instead of transforming data in the input space as above, we can apply transformations on the feature space.

- For eg., an autoencoder might be used to extract the latent representation.

iii) GAN-based augumentation: GAN have been proven to work extremely well on data generation so they are a natural choice for data augumentation.

- Benefits:

i) Improves generalization
ii) Reduces overfitting.

Page No.:
Date:

Q.8 Explain different activation functions and loss functions. How to choose the activation functions and loss functions for deep learning applications.

⇒
- Choosing an activation function and loss function is directly dependent upon the output you want to predict.
- There are different cases and different outputs of a predictive model.

1) When the output is a numerical value that you are trying to predict

⇒ Consider predicting the prices of houses provided with different features of the house.
- A neural network structure where the final layer or the output layer will consist of only one neuron that reverts the numerical value.
- Activation function to be used in such cases,
  a) Linear Activation
  b) ReLU Activation.
- Loss function to be used in such cases,
  a) MSE.

2) When the output you are trying to predict is Binary

⇒ Consider a case where the aim is to predict whether a loan applicant will default or not.
- In these types of cases, the output layer consists of only one neuron that is

responsible to result in a value that is between 0 and 2 that can be also called probablistic scores.
- For computing accuracy of the prediction, it is again compared with the true labels
- Activation function to be used in such cases,
  a) sigmoid activation
  b) Binary cross entropy.

3) Predicting a single class from many classes
=> Consider a case where you are predicting the name of the fruit amongst 5 different fruits.
- In this case, the output layer will consist of only one neuron for every class and it will revert a value between 0 and 1.
- The output is the probability distribution that results in 1 when all are added.
- Activation functions to be used in such cases,
  a) Softmax
- Loss function to be used in such cases,
  a) cross-entropy.

4) Predicting multiple labels from multiple class
=> Consider the case of predicting different objects in an image having multiple objects.
- This is termed as multiclass classification.
- In these types of cases, the output layer consists of only one neuron that is responsible to result in a value

that is between 0 and 1 that can be also called probabilistic scores.

- Activation function to be used in such cases,
  a) sigmoid
- Loss function to be used in such cases,
  a) Binary cross entropy.

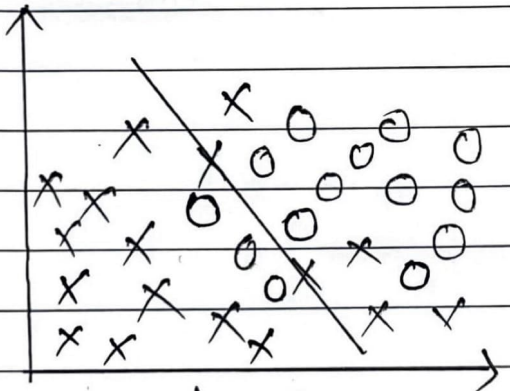
Q.9 Describe underfitting, overfitting and bias-variance trade-off.

=>

1) Underfitting

=> A statistical model or machine learning algorithm is said to have underfitting when it cannot capture the underlying trend of the data.

- Underfitting destroys the accuracy of the machine learning model.
- Its occurence simply means that our model or the algorithm does not fit the data well enough.



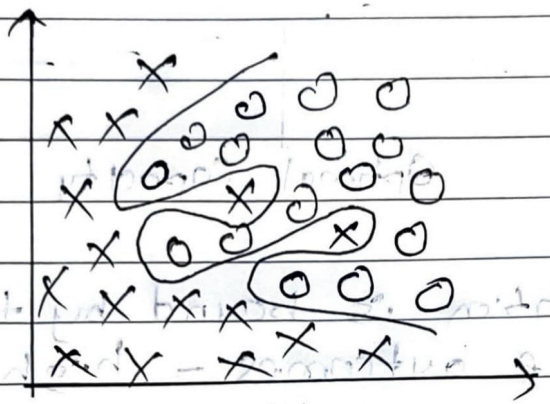
Under-fitting

- Techniques to reduce under-fitting:
  a) Increase model complexity.
  b) Remove noise from data.
  c) Increase no. of features, performing feature engineering.

2) Overfitting

=> A statistical model is said to be overfitted, when we train it with a lot of data.

- When model gets trained with so much of data, it starts learning from the noise and inaccurate data entries in our dataset.
- Then the model does not categorize the data correctly, because of too many details and noise.



Over-fitting

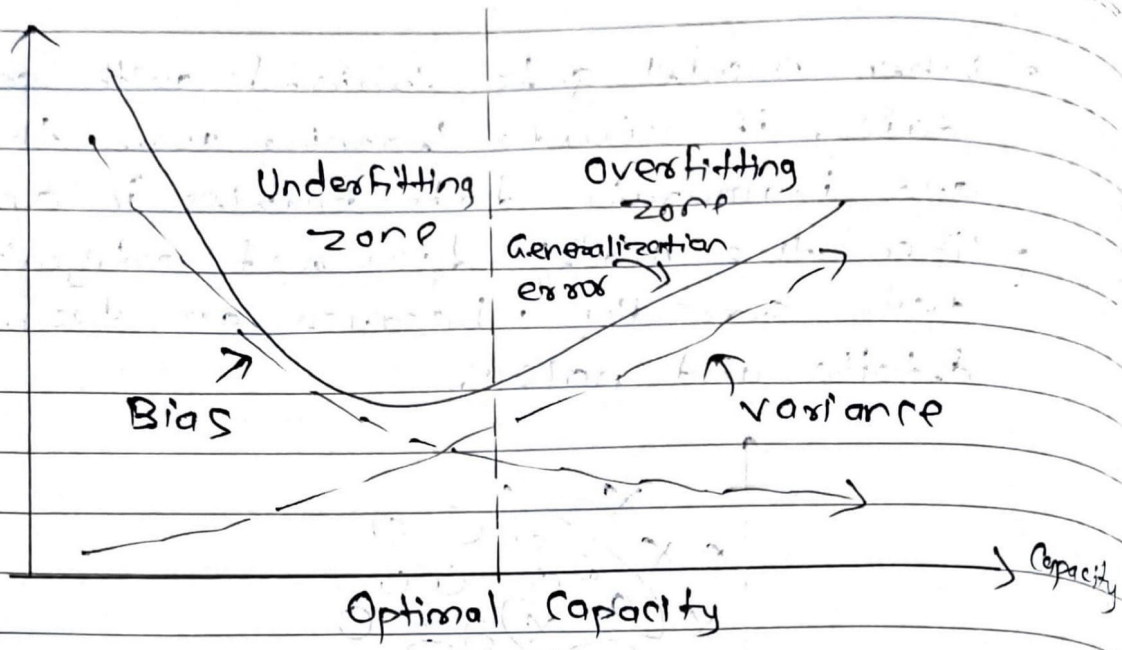
- Techniques to reduce overfitting:-

a) Increase training data.
b) Reduce model complexity.
c) L1 and L2 regularization.

3) Bias-variance trade-off

⇒ Bias and variance are two key sources of error in ML model that directly impact their performance and generalization ability.

- Bias is the error that happens when a machine learning model is too simple and doesn't learn enough details from the data.
- Variance is the error that happens when a ML model learns too much from the data, including random noise.



- Generalization is bound by the two undesirable outcomes - high bias and high variance.

Q.10 Explain the dropout method and its advantages.

=>

- Another strategy to regularize deep nairal network is dropout.
- Dropout falls into noise injection techniques and can be seen as noise injection into the hidden units of the network.
- Dropout is a regualization technique used in neural networks to prevent overfitting.
- During training, random neurons are dropped with a certain probability, so they do not participate in forward or backward propagation.

- **How it works?**

- A dropout rate (e.g., 0.5) is chosen.
- At each training iteration, 50% of the neurons in the layers are randomly turned off.
- This forces the networks to learn redundant representation, improving generalization.

- **How it solves overfitting?**

- Prevents neurons from becoming too dependent on specific other neurons.
- Encourages the network to learn more robust and generalized features.

- Act like training many smaller networks and averaging their predictions.

- Advantages:

1) Reduces overfitting
=> Helps the model generalize better to unseen data.

2) Efficient regularization.
=> Simple to implement and works well in practice.

3) Improves Robustness
=> Trains the model to rely on multiple paths of information.

Q.11 Explain optimizers. Why optimizers are required?

⇒

- An optimizer is an algorithm that adjusts the weights of neural network to minimize the loss function and improve model accuracy during training.
- Optimizers use the gradient of the loss function w.r.t each weight to decide how to updates the weights:

$$w = w - \alpha \cdot \frac{\partial L}{\partial w}$$

where,

W = weights

α = learning rate

$\frac{\partial L}{\partial w}$ = gradient loss

- Why Optimizers are Required:

1) Minimize error

⇒ Optimizers help reduce the loss between predicted and actual outputs.

2) Efficient training

⇒ They accelerate convergence, saving time and resources.

- Reach optimal weight faster.

3) Handle Complex Models

=> For deep networks with millions of parameters, optimizers intelligently adjust weights to navigate the complex loss landscape.

4) Stability in Learning

=> They help maintain stability by avoiding exploding or vanishing gradients, especially in deep architectures.